

COSC 423/523: Introduction to Artificial Intelligence

PA1: Search in Pacman

Fall 2026

100 points

10% of course grade

Due: See Canvas

Introduction

In this project, your Pacman agent will find paths through its maze world, both to reach a particular location and to collect food efficiently. You will build general search algorithms and apply them to Pacman scenarios.

This project includes an autograder that you can run on your machine:

```
python autograder.py --no-graphics
```

The code for this project consists of several Python files, some of which you will need to read and understand in order to complete the assignment, and some of which you can ignore.

Starter folder: `Student_Starter_Code/search`

Files you will edit:

- `search.py`: Where all of your search algorithms will reside.
- `searchAgents.py`: Where all of your search-based agents will reside.

Files you might want to look at:

- `pacman.py`: The main file that runs Pacman games. This file describes the Pacman `GameState` type used in this project.
- `game.py`: The logic behind how the Pacman world works. This file describes several supporting types such as `AgentState`, `Agent`, `Direction`, and `Grid`.
- `util.py`: Utility code used by the project.

Supporting files you can ignore:

- `graphicsDisplay.py`: Graphics for Pacman.
- `graphicsUtils.py`: Support for Pacman graphics.
- `textDisplay.py`: ASCII graphics for Pacman.
- `ghostAgents.py`: Agents that control ghosts.
- `keyboardAgents.py`: Keyboard interfaces for controlling Pacman.
- `layout.py`: Code for reading layout files and storing their contents.
- `autograder.py`: Project autograder.
- `testParser.py`: Parses autograder tests and solution files.
- `testClasses.py`: General autograding test classes.
- `test_cases/`: *Directory containing test cases for each question.*
- `searchTestClasses.py`: Project-specific autograding test classes.

Files to edit and submit. You will fill in portions of `search.py` and `searchAgents.py` during this assignment. Do not change the other files in the distribution.

Evaluation. Your code will be autograded for technical correctness. Do not change the names of any provided functions or classes. The correctness of your implementation is the final basis for grading, and submissions may be reviewed individually if necessary.

Collaboration. You may work individually or in a team of up to three students. Do not share code across teams. Every member of a team must be able to explain the submitted implementation.

Academic integrity. Submitted code must be your own work or the work of your registered team. Submissions may be checked against other submissions for inappropriate code similarity.

Getting help. If you are stuck, contact the course staff through the available course support channels.

Discussion. Be careful not to post assignment spoilers or solution code publicly.

Autograder note. The supplied autograder reports 18 legacy autograder points for the required questions. Gradescope uses the 100-point allocation listed below. The default command runs only the Fall 2026 required questions.

Welcome to Pacman

After opening the starter-code folder and changing to the project directory, you should be able to start a game of Pacman by running:

```
python pacman.py
```

Pacman lives in a maze of corridors and food. Navigating this world efficiently will be Pacman's first search task.

The simplest agent in `searchAgents.py` is called `GoWestAgent`, which always moves West. This agent can occasionally succeed:

```
python pacman.py --layout testMaze --pacman GoWestAgent
```

However, it will fail when the maze requires turns:

```
python pacman.py --layout tinyMaze --pacman GoWestAgent
```

If Pacman becomes stuck, you can exit the game by pressing `CTRL-c` in the terminal.

The `pacman.py` program supports several command-line options. Options can often be written in either a long form, such as `--layout`, or a short form, such as `-l`. To view the available options and their default values, run:

```
python pacman.py -h
```

Required work and grading

Question 1. Finding a Fixed Food Dot using Depth-First Search (15 PA points)

In `searchAgents.py`, you will find a fully implemented `SearchAgent`, which plans out a path through Pacman's world and then executes that path step-by-step. The search algorithms for formulating the plan are not implemented.

First, test that the provided `SearchAgent` is working correctly by running:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch
```

This command tells `SearchAgent` to use `tinyMazeSearch`, which is already implemented in `search.py`. Pacman should navigate the maze successfully.

Implement the depth-first search (DFS) algorithm in the `depthFirstSearch` function in `search.py`. To make the algorithm complete, write the graph-search version of DFS, which avoids expanding states that have already been visited.

All search functions must return a list of actions that leads the agent from the start state to the goal. Every action in the returned sequence must be a legal move.

Your code should find solutions for:

```
python pacman.py -l tinyMaze -p SearchAgent
```

```
python pacman.py -l mediumMaze -p SearchAgent
```

```
python pacman.py -l bigMaze -z .5 -p SearchAgent
```

The Pacman board will show an overlay of the states explored and the order in which they were explored. Brighter red indicates earlier exploration.

Run the autograder for this question with:

```
python autograder.py -q q1 --no-graphics
```

Question 2. Breadth-First Search (15 PA points)

Implement the breadth-first search (BFS) algorithm in the `breadthFirstSearch` function in `search.py`. Again, write a graph-search algorithm that avoids expanding states that have already been visited.

Test your code with:

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
```

```
python pacman.py -l bigMaze -p SearchAgent -a fn=bfs -z .5
```

For problems in which all actions have equal cost, BFS should find a least-cost solution.

If Pacman is moving too slowly for observation, the project supports the `--frameTime 0` option.

The search code should remain general rather than depending on a particular Pacman maze.

Run the autograder for this question with:

```
python autograder.py -q q2 --no-graphics
```

Question 3. Varying the Cost Function (15 PA points)

While BFS will find a fewest-actions path to the goal when actions have equal cost, we may want to find paths that are “best” in other senses.

Consider the `mediumDottedMaze` and `mediumScaryMaze` layouts. By changing the cost function, Pacman may prefer different paths through the maze.

Implement the uniform-cost graph search algorithm in the `uniformCostSearch` function in `search.py`.

Your implementation must work with the standard search problem as well as the different cost functions used by the provided agents.

Test your implementation with:

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs

python pacman.py -l mediumDottedMaze -p StayEastSearchAgent

python pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```

The agents in these examples all use uniform-cost search but differ in the cost functions provided to the search problem.

Run the autograder for this question with:

```
python autograder.py -q q3 --no-graphics
```

Question 4. A* Search (15 PA points)

Implement A* graph search in the `aStarSearch` function in `search.py`.

A* takes a heuristic function as an argument. Heuristics take two arguments: a state in the search problem and the problem itself. The `nullHeuristic` function in `search.py` is a trivial example.

You can test your A* implementation on the original problem of finding a path through a maze to a fixed position using the provided Manhattan-distance heuristic:

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

A* should return an optimal solution when used with an appropriate heuristic.

Run the autograder for this question with:

```
python autograder.py -q q4 --no-graphics
```

Question 5. Finding All the Corners (20 PA points)

The real power of A* becomes more apparent with a more challenging search problem.

In corner mazes, there are four dots, one in each corner. The new search problem is to find the shortest path through the maze that touches all four corners, whether or not the maze actually contains food in those locations.

For some mazes, the shortest path does not necessarily visit the closest corner first.

Complete Question 2 before working on this question, because this question is tested using breadth-first search.

Implement the `CornersProblem` search problem in `searchAgents.py`. You must choose a state representation that encodes all of the information necessary to determine whether all four corners have been reached.

Your search agent should solve:

```
python pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

```
python pacman.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

To receive full credit, define an abstract state representation that does not encode irrelevant information, such as ghost positions or unrelated food locations. In particular, do not use a Pacman `GameState` as a search state.

An instance of the `CornersProblem` class represents an entire search problem, not one particular search state. Particular states are returned by the functions you implement, and many different states may exist simultaneously while the search algorithm is running. These states must be independent of one another.

Complete the state representation, start state, goal test, and successor function in `searchAgents.py`.

Run the autograder for this question with:

```
python autograder.py -q q5 --no-graphics
```

Question 6. Corners Problem: Heuristic (20 PA points)

Complete Question 4 before working on this question, because this question uses A* search.

Implement a non-trivial, consistent heuristic for the `CornersProblem` in `cornersHeuristic`.

Test your heuristic with:

```
python pacman.py -l mediumCorners -p AStarCornersAgent -z 0.5
```

`AStarCornersAgent` runs A* search on the `CornersProblem` using `cornersHeuristic`.

Admissibility vs. consistency. Heuristics are functions that take search states and return numbers that estimate the remaining cost to a goal.

To be *admissible*, heuristic values must be non-negative lower bounds on the actual shortest remaining cost.

To be *consistent*, it must additionally be true that if an action has cost c , taking that action can cause the heuristic value to decrease by at most c .

Admissibility alone is not sufficient to guarantee correctness in graph search; the heuristic must satisfy the stronger consistency requirement.

Non-trivial heuristics. A heuristic that always returns zero is trivial. Your heuristic must provide a non-trivial estimate of the remaining cost.

The heuristic must return 0 at every goal state and must never return a negative value.

The heuristic should reduce node expansions compared with the null heuristic without causing A* to return an incorrect solution cost.

Run the autograder for this question with:

```
python autograder.py -q q6 --no-graphics
```

Optional work

Question 7. Eating All the Dots (up to 5 bonus PA1 points)

Now consider the harder search problem of eating all of the Pacman food in as few steps as possible.

The provided `FoodSearchProblem` in `searchAgents.py` formalizes the food-clearing problem. A solution is a path that collects all of the food in the Pacman world.

For this search problem, solutions do not take into account ghosts or power pellets. They depend only on the placement of walls, regular food, and Pacman.

You can run the provided food-search agent with:

```
python pacman.py -l testSearch -p AStarFoodSearchAgent
```

`AStarFoodSearchAgent` runs A* search on `FoodSearchProblem` using `foodHeuristic`.

Complete `foodHeuristic` in `searchAgents.py` with a consistent heuristic for `FoodSearchProblem`.

You can also test the agent on:

```
python pacman.py -l trickySearch -p AStarFoodSearchAgent
```

The heuristic must be non-trivial, non-negative, and consistent. It must return 0 at every goal state and must never return a negative value.

Run the optional autograder question with:

```
python autograder.py -q q7 --no-graphics
```

Question 8. Suboptimal Search (up to 5 bonus PA1 points)

Sometimes, even with A* and a good heuristic, finding an optimal path through all of the food dots can be difficult. In these cases, it may still be useful to find a reasonably good path quickly.

The provided `ClosestDotSearchAgent` repeatedly searches for a path to the closest remaining food dot. It is missing the function that finds a path to the closest dot.

Implement `findPathToClosestDot` in `searchAgents.py`.

Test the agent with:

```
python pacman.py -l bigSearch -p ClosestDotSearchAgent -z .5
```

The `ClosestDotSearchAgent` will not always find the shortest possible path through the entire maze. You should understand why repeatedly choosing the closest food dot may produce a suboptimal overall solution.

Run the optional autograder question with:

```
python autograder.py -q q8
```

Submission

Submit the Python files you edited:

- `search.py`
- `searchAgents.py`

Upload these files through Gradescope.

If you worked in a team, make one Gradescope submission and use Gradescope's group-submission feature to select all members of the team.

In addition, submit one contribution statement through Canvas describing how the work was divided among the team members.

If you worked individually, state in the Canvas contribution submission that the assignment was completed individually.

Do not submit the entire Pacman engine unless Gradescope explicitly requests additional files.